Metro Route Planning via Weighted Graph Modeling and Incremental System Enhancement

Jinghao Chen[a]

[a]University of Science and Technology of China, No. 96 Jinzhai Road, Baohe District, Hefei, 230026, Anhui, China

ARTICLE INFO

Keywords:
weighted graph
shortest path
Dijkstra algorithm
metro network
transfer-aware routing
GUI

ABSTRACT

This report studies urban metro route planning under a weighted-graph framework and documents the incremental development from the teaching template to three completed stages: the basic shortest-path solver, an enhanced graphical user interface, and an optional transfer-aware extension for Beijing. The metro system is modeled as an undirected weighted graph in which stations are nodes and inter-station travel distances are edge weights. On this basis, a hand-written Dijkstra algorithm is implemented to compute the minimum-cost route between arbitrary origin and destination stations. Compared with the original template, the completed basic task mainly fills in the graph data structure, graph construction routine, shortest-path solver, and station-name interface. On top of the baseline solver, the GUI stage further improves usability through more interactive visualization mechanisms and clearer route presentation. Finally, the optional stage introduces transfer penalties by expanding the search state from station space to station-line space, so that the path cost jointly reflects running distance and transfer overhead. In the experimental part, a Beijing case from Beigongmen to Guomao is further added to compare the distance-only route with the transfer-aware route under a transfer penalty setting of $\tau = 1.0$. The entire report emphasizes what was modified relative to the previous stage rather than restating all details of the original template.

1. Introduction

Urban metro route planning can be naturally formulated as a shortest-path problem on a weighted graph. According to the experimental requirement, the system should construct the metro network, solve the shortest path between arbitrary stations, display both the route and its total travel cost, and additionally support a transfer-aware extension for Beijing. The teaching framework already provides the program entry and a visualization interface, while the algorithmic core in the template remains incomplete. Therefore, the central purpose of this report is not to rewrite the whole framework description, but to explain the incremental modifications introduced during three stages of development.

Let the metro network be denoted by

$$G = (V, E, w),$$

where V is the station set, E is the undirected edge set, and $w : E \to \mathbb{R}_{>0}$ is the edge-weight function. For a path

$$P = (v_0, v_1, \dots, v_k),$$

its baseline travel cost is defined as

$$C(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$

Hence the basic task is equivalent to solving

$$P^* = \arg\min_{P:\, s \rightsquigarrow t} C(P).$$

The completed system is developed in three consecutive stages:

1. completion of the basic shortest-path solver from the algorithm template,
2. GUI enhancement beyond the baseline version,
3. optional transfer-aware routing for Beijing.

2. Mathematical Model and Overall Design

2.1. Graph representation

Each station corresponds to one node ID, and the adjacency-distance matrix determines whether two nodes are connected. If the matrix entry is positive, an undirected weighted edge is inserted:

$$(i, j) \in E \iff A_{ij} > 0, \qquad w(i, j) = A_{ij}.$$

Because the metro data are symmetric in the undirected setting, it is sufficient to scan the upper triangular part of the adjacency matrix during graph construction.

An adjacency-list structure is adopted for implementation. For every node u, its neighbors are stored as a dictionary

$$\mathcal{N}(u) = \{v : w(u, v)\},$$

which supports efficient edge insertion and local traversal during Dijkstra relaxation. This design is well aligned with the repeated operation

$$\text{for } v \in \mathcal{N}(u) \text{ do relax}(u, v).$$

2.2. Baseline optimization objective

Given source station s and destination station t, define the tentative shortest distance by $d(v)$. Dijkstra's method repeatedly extracts the unvisited node with smallest tentative distance and updates its neighbors by

$$d(v) \leftarrow \min\{d(v),\ d(u) + w(u, v)\}.$$

If prev(v) records the predecessor of v on the current best route, then the final path is reconstructed by backtracking from t to s.

2.3. Transfer-aware objective

For the optional problem, distance alone is no longer sufficient, because a transfer should add a fixed penalty. Suppose the route is traveled along line sequence

$$L = (\ell_0, \ell_1, \ldots, \ell_{k-1}),$$

and let $\tau > 0$ denote the transfer cost. Then the generalized objective becomes

$$C_\tau(P, L) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}) + \tau \sum_{i=1}^{k-1} \mathbf{1}(\ell_i \neq \ell_{i-1}).$$

To optimize this objective, the search state is expanded from node space to node-line space:

$$\widetilde{V} = \{(v, \ell) : \ell \in \mathcal{L}(v)\},$$

where $\mathcal{L}(v)$ is the set of lines passing through station v. The transfer-aware path problem is therefore transformed into a standard shortest-path problem on the expanded state graph.

3. Stage I: Basic Task Compared with the Code Template

The template leaves the algorithm module unfinished, so the first stage focuses on completing the mathematical core required by the README. Relative to the template, the modifications are concentrated in four places: graph storage, graph construction, Dijkstra search, and the station-name interface.

Table 1
Core modifications from the teaching template to the basic task

Module	Template status	Modification in the basic task
Graph class	Key methods left as `TODO`	Implemented node insertion, undirected weighted edge insertion, neighbor query, node count, edge count, and unique edge enumeration
`build_graph`	Returns an empty graph	Scans the station map and adjacency matrix, adds all nodes, then inserts all positive-weight undirected edges
`dijkstra`	Returns `(inf, [])`	Implements heap-based Dijkstra, predecessor recording, unreachable-state detection, and path reconstruction
`MetroSystem.shortest_path`	Only contains interface description	Converts station names to IDs and calls the completed shortest-path solver

3.1. Main modifications

3.2. Completion of the graph data structure

The first essential change is the completion of the `Graph` class. Instead of keeping only a node dictionary, the final basic version introduces an adjacency dictionary to represent weighted undirected edges. Formally, the storage invariant can be written as

$$\texttt{adj}[u][v] = w(u,v), \qquad \texttt{adj}[v][u] = w(u,v).$$

This symmetric update guarantees the correctness of an undirected metro network.

The edge count is computed by

$$|E| = \frac{1}{2}\sum_{u\in V} |\mathcal{N}(u)|,$$

which avoids double counting. Likewise, the edge listing routine keeps only one copy for each unordered pair $\{u, v\}$.

Listing 1: Representative implementation idea for the completed graph structure

```
def add_edge(self, u, v, weight=1.0):
    if u not in self.nodes:
        self.add_node(u)
    if v not in self.nodes:
        self.add_node(v)
    self.adj[u][v] = float(weight)
    self.adj[v][u] = float(weight)
```

Compared with the template, this modification turns the graph object from a placeholder into an executable weighted-network model, thereby making subsequent route search possible.

3.3. Construction of the metro graph from raw data

The second change is the completion of `build_graph`. The template only specifies the intended input-output relation, whereas the completed version explicitly maps the data file into the graph model. If station IDs start from 1 while matrix indices start from 0, then the edge insertion rule is

$$u = i + 1, \qquad v = j + 1, \qquad \text{if } A_{ij} > 0 \text{ then add } (u, v, A_{ij}).$$

Because the graph is undirected, only the range $j > i$ is scanned, which avoids repeated insertion and keeps the graph consistent with the metric matrix.

3.4. Implementation of Dijkstra's algorithm

The third major completion concerns the shortest-path solver. The template reserves the function signature but leaves the algorithm empty. The basic task adds the standard three-component mechanism:

$$\text{distance map } d, \qquad \text{predecessor map } \mathrm{prev}, \qquad \text{priority queue } Q.$$

Initially,

$$d(s) = 0, \qquad d(v) = \infty \ (v \neq s).$$

At each iteration, the node with minimum tentative distance is extracted from the heap. For every neighbor v of the current node u, the relaxation rule is

$$\text{if } d(u) + w(u, v) < d(v), \quad \text{then } d(v) \leftarrow d(u) + w(u, v),\ \mathrm{prev}(v) \leftarrow u.$$

After termination, the path is reconstructed by repeatedly applying

$$v \leftarrow \mathrm{prev}(v)$$

from t back to s.

Listing 2: Key relaxation step in the completed Dijkstra solver

```
for v, w in G.neighbors(u).items():
    if v in visited:
        continue
    new_dist = cur_dist + w
    if new_dist < dist[v]:
        dist[v] = new_dist
        prev[v] = u
        heapq.heappush(pq, (new_dist, v))
```

The resulting time complexity is

$$O\big((|V| + |E|) \log |V|\big),$$

which is suitable for metro-scale path planning under repeated user queries.

3.5. High-level interface from station names to routes

The final basic-task modification is the completion of `MetroSystem.shortest_path`. The template already loads the station map and adjacency matrix, but the actual name-based query interface is unfinished. The completed implementation uses the mapping

$$\phi : \text{station name} \mapsto \text{station ID}$$

to convert user input into graph indices, and then calls the shortest-path solver:

$$(s, t) = (\phi(\texttt{src\_name}), \phi(\texttt{dst\_name})).$$

This change connects the algorithm layer with the GUI layer and turns the program into a fully interactive route-planning system.

4. Stage II: GUI Upgrade Compared with the Basic Task

The basic task already makes the solver executable, but its role is still primarily algorithmic. The second stage therefore focuses on interface-level refinement. Relative to the baseline shortest-path version, the GUI upgrade emphasizes better interaction, clearer visual control, and more stable visualization behavior, while keeping the underlying route solver unchanged.

Table 2
GUI enhancement relative to the basic-task version

Aspect	Baseline after Stage I	GUI enhancement
View manipulation	Static display after route generation	Adds toolbar-based zoom-in, zoom-out, and fit-view operations
Canvas interaction	Limited observation of dense networks	Adds mouse-wheel zoom and mouse-drag panning for local inspection
Label readability	Fixed text size may become crowded or too small	Introduces dynamic text rescaling according to current zoom level
Display stability	Redrawing may disturb the current observation window	Preserves and restores view state during redraw operations
Visual consistency	Functional but less polished layout	Uses unified style configuration, card-like panels, and clearer status feedback

4.1. Main modifications

4.2. Interactive view control

A dense metro graph may contain many nodes and crossing edges, so a purely static canvas is not sufficient for detailed inspection. The upgraded GUI adds explicit toolbar operations and continuous mouse interaction. If the current horizontal and vertical spans are denoted by Δx and Δy, then zooming with scale factor α updates the window to

$$\Delta x' = \alpha \Delta x, \qquad \Delta y' = \alpha \Delta y.$$

When zooming around a local center (x_c, y_c), the viewport is adjusted proportionally so that the user keeps focus on the selected region. This is especially useful when the shortest path passes through crowded transfer areas.

Listing 3: Representative zoom-control logic in the upgraded GUI

```
def _zoom(self, scale_factor, center=None):
    x0, x1 = self.ax.get_xlim()
    y0, y1 = self.ax.get_ylim()
    new_width = (x1 - x0) * scale_factor
    new_height = (y1 - y0) * scale_factor
    ...
    self.ax.set_xlim(...)
    self.ax.set_ylim(...)
```

4.3. Dynamic label scaling

Once free zoom is supported, a fixed font size becomes visually unstable: labels can overlap when zoomed out and become unnecessarily tiny when zoomed in. To address this, the upgraded GUI adjusts label size according to the current zoom factor z through a monotone transformation of the form

$$\eta(z) = \mathrm{clip}(c z^{\beta}, \eta_{\min}, \eta_{\max}),$$

where $c > 0$ is the base size, $\beta \in (0, 1)$ is a smoothing exponent, and the clipping operation keeps the font within a reasonable range. In the implementation, station labels and endpoint labels are scaled separately so that the source and destination remain visually salient.

4.4. Stable redraw and improved usability

The GUI upgrade also improves usability at the redraw level. Let

$$S = (x_{\min}, x_{\max}, y_{\min}, y_{\max})$$

be the current view state. Before a redraw, the program stores S and restores it afterward, which avoids abrupt jumps in the viewport when the user changes stations or recomputes a path. In addition, the side panel uses clearer grouping for city selection, station selection, and route output, while the status bar provides immediate textual feedback on loading and solving states.

These modifications do not alter the shortest-path mathematics, but they substantially improve the practical usability of the solver and make later optional extensions easier to operate and demonstrate.

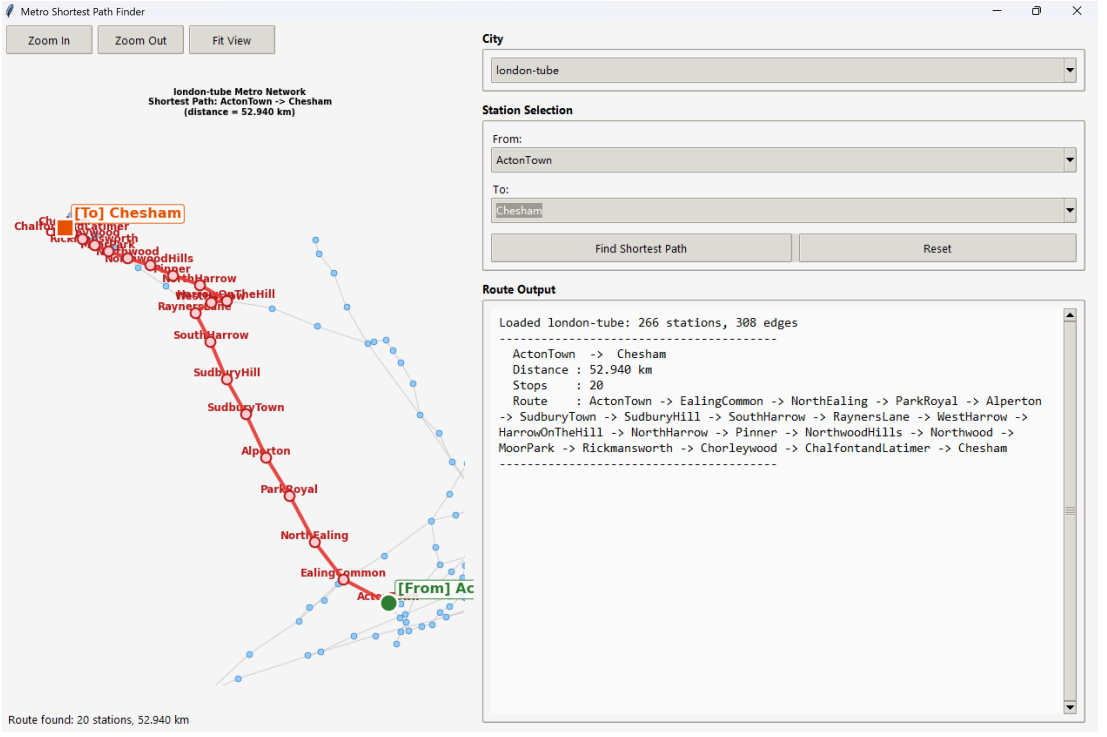

Figure 1: Enhanced GUI for metro route planning with interactive zooming, panning, route output, and improved control layout.

5. Stage III: Optional Task Compared with the GUI Upgrade

The final stage extends the system from distance-only routing to transfer-aware routing. According to the requirement, this part only needs to be solved for Beijing, because only Beijing provides line information. Relative to the GUI-upgraded version, this stage changes both the optimization model and the interface logic.

5.1. Main modifications

5.2. Loading and organizing line information

The first new element is the station-line file. The optional stage adds a parser that maps each station name to its incident line set:

$$\mathcal{L}(v) = \{\ell_1, \ell_2, \dots\}.$$

After station names are converted to station IDs, the line information is reorganized as

$$\mathcal{L} : V \to 2^{\text{Lines}},$$

which becomes the basis of transfer detection.

Table 3
Optional transfer-aware extension relative to the GUI-upgraded version

Module	GUI-upgraded version	Optional extension
Input data	Uses station map and distance matrix	Further loads station-line data for Beijing
State definition	Path search in station space V	Path search in expanded station-line space $\widetilde{V}$
Cost function	Pure distance accumulation	Distance plus fixed transfer penalty
Routing interface	`shortest_path(src,dst)`	Adds `use_transfer_time` and `transfer_time` parameters
GUI controls	General route query only	Adds Beijing-only transfer checkbox and editable transfer-cost input

5.3. From station states to station-line states

Under transfer penalties, the current cost depends not only on which station has been reached, but also on which line is currently being used. Therefore, the optional task replaces the original state v by the expanded state

$$(v, \ell).$$

Suppose the current state is (u, ℓ) and the search considers a neighboring station v. If the two stations share candidate lines

$$\mathcal{L}(u) \cap \mathcal{L}(v),$$

then each feasible next line $\ell' \in \mathcal{L}(u) \cap \mathcal{L}(v)$ induces a transition

$$(u, \ell) \to (v, \ell').$$

The corresponding transition cost is

$$\widetilde{w}\big((u, \ell), (v, \ell')\big) = w(u, v) + \tau \mathbf{1}(\ell' \neq \ell).$$

Hence the transfer problem becomes a standard shortest-path problem on the expanded graph.

Listing 4: Core transition rule of the transfer-aware solver

```
for v, w in G.neighbors(u).items():
    common_lines = station_lines.get(u, set()) & station_lines.get(v, set())
    for next_line in common_lines:
        penalty = 0.0 if next_line == cur_line else float(transfer_time)
        new_dist = cur_dist + float(w) + penalty
        ...
```

Compared with the baseline Dijkstra solver, the main conceptual difference is that route quality is no longer determined solely by geometric distance. Instead, it is governed by a composite objective that balances running distance and transfer overhead.

5.4. Path reconstruction and compatibility

Although the search is performed in the expanded state space, the final output shown to the user should still be a station sequence. Therefore, after the optimal state path

$$((v_0, \ell_0), (v_1, \ell_1), \ldots, (v_m, \ell_m))$$

is found, the implementation compresses it back into a station path by removing consecutive repeated station IDs. This keeps the GUI output format compatible with the previous stage while allowing the internal solver to remain transfer-aware.

If no line information is available, the system falls back to the original distance-only Dijkstra routine. Thus the optional extension preserves backward compatibility with non-Beijing cities.

Table 4
Graph scale of the loaded metro systems

City	Number of stations	Number of edges
Barcelona	136	160
Beijing	126	139
Berlin	173	184

5.5. GUI support for the optional problem

The README explicitly states that the optional interface is not provided by the original framework, so the GUI must also be extended. The optional stage therefore adds a Beijing-specific control group consisting of

1. a checkbox that turns transfer-aware routing on or off,
2. an editable input box for the transfer cost τ,
3. route-output labels that switch from *Distance* to *Total Cost* when transfer penalties are enabled.

This interface design is mathematically meaningful: when transfer mode is off, the optimized objective is $C(P)$; when transfer mode is on, the optimized objective is $C_\tau(P, L)$. The GUI now makes this modeling distinction explicit to the user.

6. Complexity Discussion

For the basic task, graph construction from an $n \times n$ adjacency matrix costs

$$O(n^2),$$

because the matrix must be scanned once. The heap-based Dijkstra solver then runs in

$$O\big((|V| + |E|) \log |V|\big).$$

For the optional task, let $|\mathcal{L}|$ denote the average number of feasible line states per station. The expanded state-space solver operates on approximately $|V||\mathcal{L}|$ states, so its complexity can be expressed as

$$O\big((|\widetilde{V}| + |\widetilde{E}|) \log |\widetilde{V}|\big).$$

Although this is larger than the baseline complexity, it remains practical because the metro line set is small compared with the station set.

7. Experimental Results

To complement the implementation analysis, this section summarizes three representative shortest-path experiments obtained from the completed system. The currently available outputs include both baseline distance-based routing results and one Beijing optional comparison under transfer-aware routing. Therefore, the first three subsections report the baseline solver results for Barcelona, Beijing, and Berlin, while an additional subsection is inserted afterward to show how the optional objective changes the Beijing route from Beigongmen to Guomao.

7.1. Network scale statistics

Before route queries are issued, each metro system is first converted into an undirected weighted graph. Table 4 reports the graph sizes loaded by the program. These results verify that the completed `build_graph` routine successfully transforms the raw adjacency data into executable metro networks.

The three systems differ in graph size, but all of them remain sparse in the sense that

$$|E| = O(|V|).$$

Table 5
Summary of representative shortest-path results

City	Source station	Destination station	Distance (km)	Stops
Barcelona	BadalonaCentre	TorreBaro-Vallbona	14.181	19
Beijing	Beigongmen	Liyuan	43.650	32
Berlin	Breitenbachplatz	Samariterstrasse	15.713	25

Table 6
Detailed station sequences of the computed optimal routes

City	Optimal route
Barcelona	BadalonaCentre → PepVentura → Gorg → SantRoc → Artigues-SantAdria → Verneda → LaPau → SantMarti → BacdeRoda → Clot → Navas → Sagrera → LaSagrera → Maragall → Llucmajor → ViaJulia → TrinitatNova → Casadel'Aigua → TorreBaro-Vallbona
Beijing	Beigongmen → Xiyuan → YuanmingyuanPark(OldSummerPalace) → EastGateofPekingUniversity → Zhongguancun → HaidianHuangzhuang → Zhichunli → Zhichunlu → Dazhongsi → Xizhimen → Jishuitan → GulouDajie → Andingmen → YonghegongLamaTemple → Dongzhimen → DongsiShitiao → Chaoyangmen → Jianguomen → Yonganli → Guomao → Dawanglu → Sihui → SihuiEast → Gaobeidian → CommunicationUniversityofChina → Shuangqiao → Guanzhuang → Baliqiao → TongzhouBeiyuan → Guoyuan → Jiukeshu → Liyuan
Berlin	Breitenbachplatz → RuedesheimerPlatz → HeidelbergerPlatz → FehrbellinerPlatz → Hohenzollernplatz → Spichernstrasse → AugsburgerStrasse → Wittenbergplatz → Nollendorfplatz → Kurfuerstenstrasse → Gleisdreieck → Mendelssohn-Bartholdy-Park → PotsdamerPlatz → Mohrenstrasse → Stadtmitte → Hausvogteiplatz → Spittelmarkt → MaerkischesMuseum → Klosterstrasse → Alexanderplatz → Schillingstrasse → StrausbergerPlatz → Weberwiese → FrankfurterTor → Samariterstrasse

This sparsity pattern is consistent with real metro networks and supports efficient shortest-path computation by heap-based Dijkstra search. In addition, the average node degrees

$$\bar{d} = \frac{2|E|}{|V|}$$

are approximately 2.35 for Barcelona, 2.21 for Beijing, and 2.13 for Berlin, which further confirms that the constructed networks are sparse and well suited to adjacency-list traversal.

7.2. Shortest-path query results

Table 5 summarizes one representative origin-destination query for each city. For a returned path

$$P = (v_0, v_1, \dots, v_k),$$

the reported distance is

$$C(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}),$$

and the number of stops equals $k + 1$, namely the number of stations appearing in the final route sequence.

From Table 5, the Beijing query produces both the longest path and the largest station count among the three examples, while the Barcelona and Berlin cases are shorter despite their different network sizes. This indicates that the query result depends not only on the size of the whole graph but also on the relative topological position of the chosen endpoints.

7.3. Detailed route sequences

Because the complete station sequence is an essential part of the experimental output, Table 6 records the explicit optimal routes returned by the solver. The wrapped layout is intentionally adopted to preserve readability in the single-column format required by the report template.

These results confirm that the completed shortest-path module is able to return not only the scalar objective value but also the full station-level route needed by the GUI and by later qualitative discussion.

Table 7
Beijing optional comparison for the route from Beigongmen to Guomao

Routing mode	Objective value	Stops	Route
Distance-only	24.385	20	Beigongmen → Xiyuan → YuanmingyuanPark(OldSummerPalace) → EastGateofPekingUniversity → Zhongguancun → HaidianHuangzhuang → Zhichunli → Zhichunlu → Dazhongsi → Xizhimen → Jishuitan → GulouDajie → Andingmen → YonghegongLamaTemple → Dongzhimen → DongsiShitiao → Chaoyangmen → Jianguomen → Yonganli → Guomao
Transfer-aware ($\tau = 1.000$)	25.409	23	Beigongmen → Xiyuan → YuanmingyuanPark(OldSummerPalace) → EastGateofPekingUniversity → Zhongguancun → HaidianHuangzhuang → Zhichunli → Zhichunlu → Xitucheng → Mudanyuan → Jiandemen → Beitucheng → Anzhenmen → HuixinxijieNankou → Shaoyaoju → Taiyanggong → Sanyuanqiao → Liangmaqiao → AgriculturalExhibitionCenter → Tuanjiehu → Hujialou → Jintaixizhao → Guomao

7.4. Beijing optional comparison: distance-only versus transfer-aware routing

To further verify the optional extension, the same Beijing origin-destination pair

`Beigongmen` → `Guomao`

is tested under both objectives. When transfer penalties are disabled, the solver minimizes pure running distance $C(P)$. When transfer penalties are enabled with

$$\tau = 1.000,$$

the solver instead minimizes the generalized cost $C_\tau(P, L)$. Table 7 summarizes the two outputs returned by the program.

The comparison shows that the optional solver does not simply reproduce the distance-only route. Instead, after transfer penalties are introduced, the algorithm selects a different line sequence in order to optimize the composite objective

$$C_\tau(P, L) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}) + \tau \sum_{i=1}^{k-1} \mathbf{1}(\ell_i \neq \ell_{i-1}).$$

For this example, the pure-distance route contains 20 stations and has total geometric length 24.385 km, whereas the transfer-aware route contains 23 stations and has reported total cost 25.409 under the transfer-cost setting $\tau = 1.000$. This confirms that once line-state information is incorporated, the optimal route depends on both inter-station distance and transfer behavior rather than distance alone.

7.5. Remarks on GUI and optional extensions

The GUI upgrade is reflected not only through better path inspection and route presentation but also through its support for the optional Beijing interface. Figure 1 gives a representative snapshot of the upgraded interface, including toolbar-based view control, a structured side panel, and the route-output area. In addition, Figures 2 and 3 visualize the two Beijing outputs for the same origin-destination pair, making it easier to compare the route change caused by the

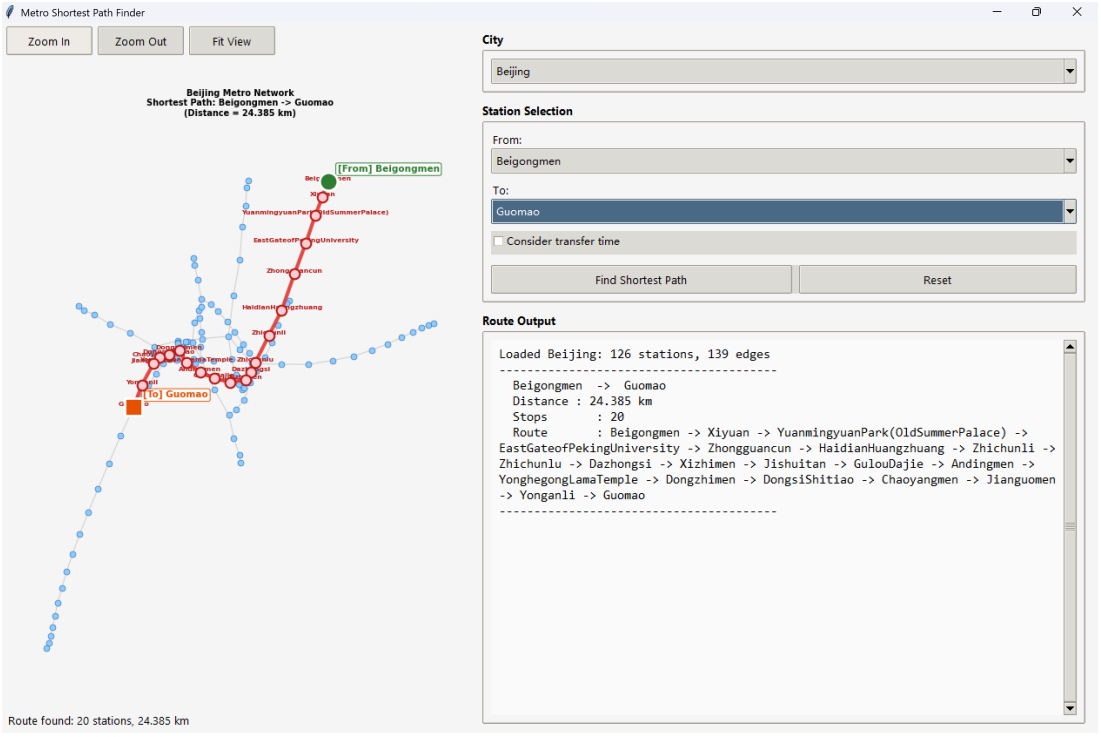

Figure 2: Distance-only Beijing routing result from Beigongmen to Guomao.

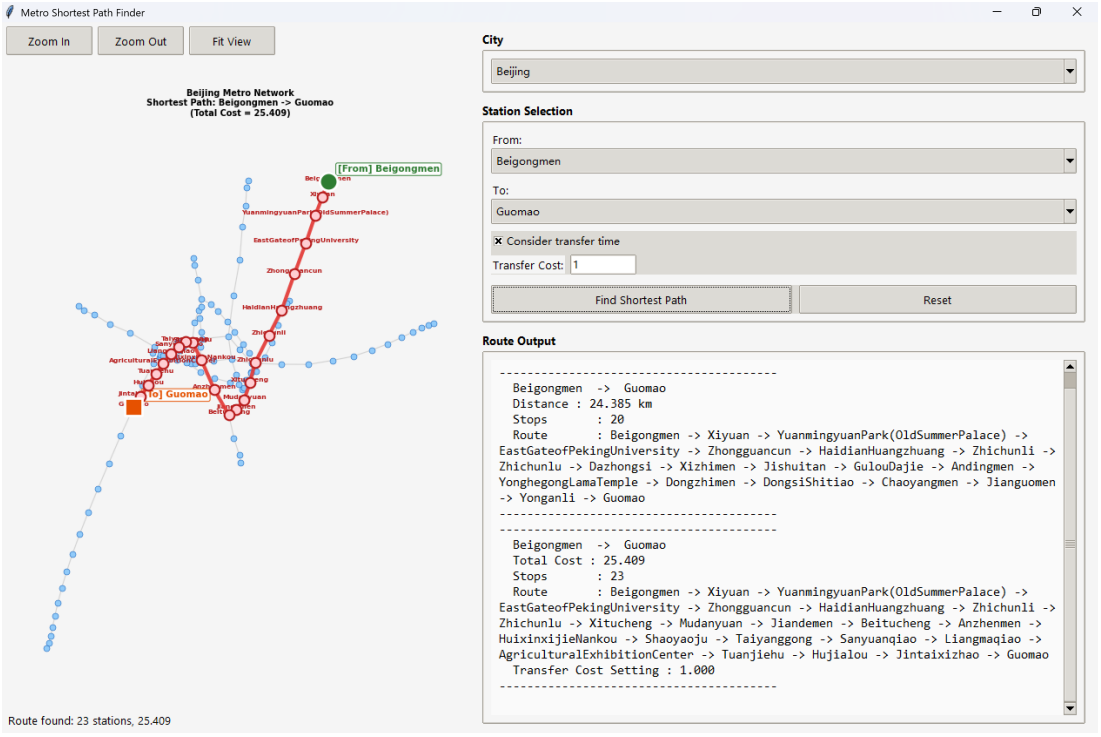

Figure 3: Transfer-aware Beijing routing result from Beigongmen to Guomao with transfer cost $\tau = 1.000$.

transfer-aware objective. The present results therefore already demonstrate the practical effect of switching between

$$C(P) \quad \text{and} \quad C_\tau(P, L) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}) + \tau \sum_{i=1}^{k-1} \mathbf{1}(\ell_i \neq \ell_{i-1}),$$

within the same software framework.

8. Conclusion

This report documents the complete development path from the teaching template to a functional and extended metro route-planning system. The first stage transforms the unfinished algorithm template into an executable weighted-graph solver by implementing the graph structure, graph construction, Dijkstra algorithm, and name-based query interface. The second stage improves usability through GUI enhancement, especially in view manipulation, readability, and interaction stability. The third stage further generalizes the routing model by introducing transfer penalties and a Beijing-specific station-line state expansion. The added Beijing comparison from Beigongmen to Guomao also shows that the optional solver can produce a route different from the pure-distance optimum once transfer cost is included. From a mathematical-modeling perspective, the project evolves from a standard shortest-path formulation to a more realistic multi-factor routing formulation while maintaining a clear incremental software architecture.